

Legacy Code Extractor

Agent Extractor — Compare LLM & Local Extraction

Paste COBOL or other program code:

```
01 WS-RECEIVED-QTY PIC 9(5).
01 WS-TIMESTAMP PIC X(26).

PROCEDURE DIVISION.

ACCEPT WS-FACILITY-ID.
ACCEPT WS-SHIPMENT-ID.
ACCEPT WS-ITEM-ID.
ACCEPT WS-RECEIVED-QTY.

EXEC SQL
UPDATE INVENTORY
SET QUANTITY = QUANTITY + :WS-RECEIVED-QTY
WHERE FACILITY_ID = :WS-FACILITY-ID
AND ITEM_ID = :WS-ITEM-ID
END-EXEC.

IF SQLCODE NOT = 0
DISPLAY "Inventory update failed for Item: " WS-ITEM-ID
STOP RUN
END-IF

EXEC SQL
INSERT INTO RECEIVING_LOG (
FACILITY_ID, SHIPMENT_ID, ITEM_ID, RECEIVED_QTY, RECEIVED_TS
)
VALUES (
```

Language: COBOL

Extract via Agent (LLM)

Extract via Local Parser

Store in Vector DB

Search Vector Database:

Enter search query to find similar requirements...

Search Requirements

Agent (LLM) Extraction



Business Rules and Requirements

Data Input

- The program accepts the following inputs:
 - Facility ID:** A 10-character alphanumeric code.
 - Shipment ID:** A 12-character alphanumeric code.
 - Item ID:** A 10-character alphanumeric code.
 - Received Quantity:** A numeric value with a maximum of 5 digits.

Inventory Update

- Requirement:** Upon receiving a shipment, the program **MUST** update the `INVENTORY` table to reflect the received quantity.
- Rule:** The `QUANTITY` in the `INVENTORY` table for the specified `FACILITY_ID` and `ITEM_ID` **MUST** be increased by the `WS-RECEIVED-QTY`.
- Error Handling:** If the inventory update fails (`SQLCODE` not equal to 0), the program **MUST** display an error message indicating the failure and the affected `ITEM_ID`, and then terminate.

Receiving Log

- Requirement:** The program **MUST** record all receiving events in the `RECEIVING_LOG` table.
- Rule:** A new record **MUST** be inserted into the `RECEIVING_LOG` table with the following information:
 - `FACILITY_ID`: The facility ID from the input.
 - `SHIPMENT_ID`: The shipment ID from the input.
 - `ITEM_ID`: The item ID from the input.
 - `RECEIVED_QTY`: The received quantity from the input.
 - `RECEIVED_TS`: The current timestamp.
- Success/Error Messaging:**
 - If the log insertion fails (`SQLCODE` not equal to 0), the program **MUST** display an error message including the affected `SHIPMENT_ID`.
 - If the log insertion is successful, the program **MUST** display a success message "Receiving recorded successfully."

Local Extracti

No results yet

RAG for Business Current State and Legacy Code also Future State

Knowledge Base Explorer

Back to Agent Extractor

Search Query

Enter keywords or describe what you're looking for. The system will search the knowledge base and show you similar requirements along with their source code.

receiving new items

Search Knowledge Base

Clear

Search Results

Score: 0.351 | ## Business Rules: ***Inventory Update:** When items are received, the 'QUANTITY' in the 'INVENTORY' table is increa...

Current State (From Knowledge Base)

```
## Business Rules:

* **Inventory Update:** When items are received, the 'QUANTITY' in the
'INVENTORY' table is increased by the 'RECEIVED_QTY' for the corresponding
'FACILITY_ID' and 'ITEM_ID'.
* **Receiving Log:** Each receipt of items must be logged in the
'RECEIVING_LOG' table.
* **Error Handling:** If the update of 'INVENTORY' fails, the program
terminates. If the insertion into 'RECEIVING_LOG' fails, a message is
displayed.

## Business Requirements:

* **Data Capture:** The system must capture the following data for each
receipt:
  * 'FACILITY_ID' (10 characters)
  * 'SHIPMENT_ID' (12 characters)
  * 'ITEM_ID' (10 characters)
  * 'RECEIVED_QTY' (Numeric, 5 digits)
* **Inventory Tracking:** The system must maintain an accurate count of
inventory by updating the 'INVENTORY' table upon receiving new items.
* **Audit Trail:** The system must maintain a log of all receipts in the
'RECEIVING_LOG' table, including:
  * 'FACILITY_ID'
  * 'SHIPMENT_ID'
  * 'ITEM_ID'
```

Source Code

```
IDENTIFICATION DIVISION.
PROGRAM-ID. DF-RECEIVING.

ENVIRONMENT DIVISION.
DATA DIVISION.
WORKING-STORAGE SECTION.

EXEC SQL INCLUDE SQLCA END-EXEC.

01 WS-FACILITY-ID      PIC X(10).
01 WS-SHIPMENT-ID      PIC X(12).
01 WS-ITEM-ID          PIC X(10).
01 WS-RECEIVED-QTY     PIC S(5).
01 WS-TIMESTAMP        PIC X(26).

PROCEDURE DIVISION.

ACCEPT WS-FACILITY-ID.
ACCEPT WS-SHIPMENT-ID.
ACCEPT WS-ITEM-ID.
ACCEPT WS-RECEIVED-QTY.

EXEC SQL
  UPDATE INVENTORY
  SET QUANTITY = QUANTITY + :WS-RECEIVED-QTY
```

Analysis Insights

Similarity Score: 35.1%

Source Language: Unknown

Recommendation: Low similarity, but may still offer insights into alternative approaches or structures.

Future State Generation

Generate Future State

Business Constraints

Real-Time Inventory Updates:

Inventory data must be updated immediately upon delivery or transaction to reflect accurate stock levels across systems.

Accurate On-Demand Reporting:

The reporting system must provide up-to-date inventory summaries at any time, matching or exceeding the accuracy of legacy batch reports.

Legacy System Compatibility:

Certain downstream systems still rely on DB2-hosted tables; data must be synchronized reliably from Azure Cloud to DB2.

Data Integrity Across Stages:

All stages—from ingestion to posting—must preserve data accuracy, enforce validation rules, and support auditability.

Event-Driven Processing:

Business logic must respond to delivery and inventory events in near real-time, ensuring timely updates and downstream consistency.

Architecture & Target Preferences

Cloud Platform:

Microsoft Azure is the primary cloud environment.

Core Components:

Azure Function Apps: Used for modular processing stages including ingestion, validation, enrichment, costing, posting, and outbound sync.
Azure Service Bus: Facilitates event-driven communication between Function Apps via message triggers.
Azure Cosmos DB: Serves as the primary operational data store for real-time inventory and delivery data.
Azure Blob Storage: Used for storing raw data inputs and message payloads.
DB2 (On-Prem or Hybrid): Legacy database still consumed by downstream systems; receives synchronized data from Azure outbound modules.

Processing Stages:

1. Ingestion & Validation: Initial data capture and rule enforcement.
2. Enrichment: Business logic application and contextual data enhancement.
3. Pre-Costing & Costing: Calculation of delivery costs and inventory valuation.
4. Posting: Finalization of transactions and inventory updates.
5. Outbound Sync: Data synchronization from Azure (Cosmos) to DB2 for legacy consumption.

Data Flow:

Service Bus messages trigger Azure Functions at each stage.
Data is transformed and validated before being persisted or forwarded.
Final inventory updates are pushed to DB2 for legacy reporting and downstream access.

Generate Future State Recommendations

Generated Future State

architecture based on Azure Functions, Service Bus, and Cosmos DB. This will enable real-time inventory updates, improve scalability, enhance auditability, and facilitate data synchronization with the legacy DB2 system, addressing the business constraints and leveraging the preferred technology stack. The transformation will be phased to minimize disruption and ensure a smooth transition.

Current State Assessment

Strengths:

- Meets basic functional requirements: Successfully captures data and updates inventory and receiving logs.
- Simple implementation: Straightforward COBOL code is easy to understand and maintain (in its current limited scope).

Weaknesses:

- Tight Coupling: Direct database updates in the COBOL program create tight coupling, making it difficult to modify or extend the functionality.

Agentic AI / API

Requirements Extraction API 1.0 OAS 3.1

/openapi.json

default ^

POST

/agent-extract

Agent Extract Api

▼

POST

/store-in-vector-db

Store In Vector Db Api

▼

POST

/search-vector-db

Search Vector Db Api

▼

GET

/

Root

▼

POST

/upload-cobol

Upload and analyze COBOL file

▼

POST

/search-requirements

Search for similar requirements

▼

GET

/list-all-requirements

Get all extracted requirements

▼

POST

/analyze-text

Analyze COBOL code from text input

▼

GET

/health

Health check

▼

GET

/debug-vector-data

Debug Vector Data

▼

POST

/generate-future-state

Generate Future State Api

▼

POST

/generate-future-state-summary

Generate Future State Summary Api

▼

GET

/stats

Get extraction statistics

▼

Schemas ^

Body_upload_cobol_file_upload_cobol_post >

Expand all

object

HTTPValidationError >

Expand all

object

ValidationError >

Expand all

object